RuboCop and Editors



If you are using an integrated development environment (IDE) or coding editor, there's likely an extension that will read your RuboCop configuration and display RuboCop issues in your editor as you type. You may even be able to auto-correct from the editor without running the command line tool. (This goes for Standard as well.)

Each file gets one character. If the file has no issues, it is represented with a dot. We have, apparently, an issue in each file, so we have no dots. If RuboCop does flag something, the file is represented by the most serious issue. Each individual cop specifies its own severity level. Following are RuboCop's severity levels in order of less serious to most serious:

- `I` is used for "info." It's rare for an "info" failure to be reported. Typically an "info" failure will not be counted as a failure of the entire run as a whole.
- `R` is used for "refactor," as in "something you might want to refactor."
- `C` is used for "convention," meaning a style issue that is part of your team's conventions, rather than something that might objectively be a problem for all teams. Cops whose names don't start with `Lint` default to this severity.
- `W` is used for "warning," meaning the file is legal Ruby but has something that you might consider an "objective" problem. The indentation is unbalanced, there's some ambiguity, something like that. Cops whose names start with `Lint` default to this severity.
- `E` is used for "error," meaning the file is not legal Ruby.
- `F` is used for "fatal," meaning the file has a syntax error that prevents it from being parsed.

After the files comes a listing of all the offenses. The example code has a lot of basically identical offenses; we've chosen to include the display of only one of them. The layout is:

- The location of the issue, `Gemfile:3:8` gives the file name, line number, and character where the issue begins.
- The severity of the issue, which in this case is level `C`.
- If RuboCop can autocorrect the issue, it says so with `[Correctable]`.
- The name of the cop, which in this case is `Style/StringLiterals`, and a description of what the cop shows.
- The line of code with the text at issue underlined.

After the list of all the offenses is a summary line: `14 files inspected, 108 offenses detected, 102 offenses auto-correctable`. Plus, RuboCop helpfully tells us that we could also install plugins for `rubocop-rake` and `rubocop-rspec`.

That's a lot of offenses. We can get a sense for what's going on by using a different formatter from the command line:

```
rubocop --format offenses
 14/14 files |=====| Time: 00:00:00

90   Style/StringLiterals
6    Style/FrozenStringLiteralComment
4    Style/WordArray
3    Style/Documentation
1    Gemspec/RequiredRubyVersion
1    Metrics/MethodLength
1    Style/AccessModifierDeclarations
1    Style/MutableConstant
```